

Résoudre des équations

Repérer une racine, choisir une méthode numérique
et résoudre un système linéaire

OBJECTIF

Passer d'un problème chimique à une équation $f(x) = 0$ ou à un système $AX = B$, puis obtenir une solution numérique et la contrôler.

Le logiciel calcule une solution approchée. L'étudiant reste responsable du modèle, du domaine physique, de la valeur de départ et de la vérification finale.

LES SIX SCRIPTS

Script	Notion	Usage
01_reperer_une_racine.py	Tracer $f(x)$ et voir où elle coupe 0	Étape préalable
02_dichotomie.py	Programmer la dichotomie	Méthode robuste
03_methode_de_newton.py	Programmer Newton	Méthode rapide
04_fonctions_scipy.py	Utiliser bisect et newton	Calcul courant
05_systeme_lineaire.py	Utiliser <code>numpy.linalg.solve</code>	Deux espèces absorbantes
06_equilibre_fe_scn.py	Résoudre un équilibre chimique	TP6 - FeSCN

Avant tout calcul

1. Définir l'inconnue et son unité. 2. Écrire l'équation. 3. Déterminer le domaine physique possible. 4. Tracer ou évaluer f sur ce domaine. 5. Calculer. 6. Remplacer la solution dans le modèle.

1. Repérer une racine et utiliser la dichotomie

Résoudre une équation revient à chercher une valeur de x pour laquelle $f(x) = 0$. Le graphique sert à repérer un intervalle plausible, pas à fournir seul une valeur précise.

```
def f(x):  
    return x**3 - x - 1  
  
plt.plot(x, y)  
plt.axhline(0, color="black")
```

Condition de départ

Pour une fonction continue, si $f(a)$ et $f(b)$ sont de signes opposés, au moins une racine se trouve entre a et b . Dans l'exemple, le graphique conduit à choisir $[1 ; 2]$.

Principe de la dichotomie

```
while droite - gauche > precision:  
    milieu = (gauche + droite) / 2  
    if f(gauche) * f(milieu) <= 0:  
        droite = milieu  
    else:  
        gauche = milieu
```

À chaque tour, l'intervalle est partagé en deux. On conserve la moitié dans laquelle le changement de signe est présent. La largeur finale de l'intervalle contrôle la précision sur la position de la racine.

Ce qu'il faut contrôler

Contrôle	Pourquoi ?
$f(\text{gauche}) \times f(\text{droite}) \leq 0$	Vérifie le changement de signe demandé par la méthode.
$\text{droite} - \text{gauche} > \text{précision}$	Donne une condition d'arrêt simple.
$f(\text{racine})$ proche de 0	Vérifie la solution dans l'équation initiale.
racine dans le domaine physique	Écarte une solution mathématique impossible en chimie.

Un changement de signe ne garantit pas qu'il n'existe qu'une seule racine dans l'intervalle. Le graphique et l'étude du modèle restent indispensables.

2. Méthode de Newton et fonctions SciPy

La méthode de Newton remplace successivement la courbe par sa tangente. Elle demande une valeur de départ et, dans sa version programmée ici, l'expression de la dérivée.

```
x = 1.5
ecart = 1.0

while ecart > precision:
    nouveau_x = x - f(x) / derivee(x)
    ecart = abs(nouveau_x - x)
    x = nouveau_x
```

Comparer les deux méthodes

Méthode	Donnée nécessaire	Atout	Vigilance
Dichotomie	Intervalle avec changement de signe	Robuste et facile à suivre	Ne détecte pas une racine sans changement de signe
Newton	Valeur de départ ; dérivée pour le script 03	Convergence souvent rapide	Peut échouer ou partir vers une autre racine

Utiliser les outils fournis

```
from scipy.optimize import bisect, newton

racine_1 = bisect(f, 1, 2)
racine_2 = newton(f, 1.5)
```

Les spécifications à retenir

bisect(f, a, b) reçoit la fonction et les deux bornes. **newton(f, x0)** reçoit la fonction et une valeur initiale. Dans les deux cas, on conserve le contrôle **f(racine)** et le contrôle physique.

Si Newton ne converge pas, ne pas multiplier les essais au hasard. Revenir au graphique, vérifier le domaine, puis choisir une valeur de départ mieux justifiée ou utiliser la dichotomie.

Petit exercice

Remplacer la fonction par **$f(x) = x^2 - 2$** , chercher la racine positive et comparer le résultat à la valeur donnée par la calculatrice.

3. Résoudre un système linéaire

Un système de n équations linéaires à n inconnues s'écrit sous la forme $\mathbf{AX} = \mathbf{B}$. La matrice A contient les coefficients, X les inconnues et B les valeurs mesurées ou imposées.

Exemple : deux espèces absorbantes

À deux longueurs d'onde, les absorbances dépendent des concentrations c_1 et c_2 . Les deux relations sont linéaires :

$$\begin{aligned} 1200 c_1 + 200 c_2 &= 0.50 \\ 300 c_1 + 900 c_2 &= 0.39 \end{aligned}$$

Traduction dans Python

```
import numpy as np

A = np.array([[1200, 200],
              [300, 900]])
B = np.array([0.50, 0.39])

X = np.linalg.solve(A, B)
```

Lire le résultat

$X[0]$ contient c_1 et $X[1]$ contient c_2 . L'ordre des inconnues doit être identique dans chaque ligne de A . Chaque ligne de A doit correspondre à la même ligne de B .

Vérifier le système

```
print(A @ X)
print(B)
```

Les deux affichages doivent être égaux à l'arrondi numérique près. Il faut également vérifier que les concentrations calculées sont compatibles avec le contexte expérimental.

Une erreur dans l'ordre des coefficients produit souvent un résultat numérique sans message d'erreur. Écrire d'abord les équations sur papier, puis construire A et B ligne par ligne.

Quand utiliser solve ?

Lorsque les équations sont linéaires, indépendantes, et aussi nombreuses que les inconnues. Une équation contenant un carré, une exponentielle ou un produit d'inconnues n'appartient pas directement à ce cadre.

4. Applications aux TP et validation

TP ou contexte	Problème numérique possible	Script
TP6 - Complexe FeSCN	Calculer l'avancement d'équilibre pour une valeur de K	06
TP2 ou TP6 - Spectrophotométrie	Séparer deux contributions mesurées à deux longueurs d'onde	05
TP1, TP8, TP9 - Titrages	Résoudre un modèle acido-basique fourni	01 à 04 à adapter
TP11 - Séparation acido-basique	Étudier une répartition ou une équation de pH fournie	01 à 04 à adapter
Exercice générique	Résoudre $f(x) = 0$ avec une précision donnée	01 à 04

Application FeSCN

Dans le script 06, l'inconnue x est la concentration du complexe formé. Le domaine physique est $0 \leq x < \min(c_{\text{Fe}}, c_{\text{SCN}})$. La borne supérieure est légèrement diminuée afin d'éviter une division par zéro.

```
def equation(x):  
    fe_eq = concentration_fe - x  
    scn_eq = concentration_scn - x  
    return x / (fe_eq * scn_eq) - K
```

Erreurs fréquentes

- Oublier de transformer l'équation en $f(x) = 0$.
- Choisir des bornes sans examiner le graphique ou le domaine physique.
- Utiliser Newton sans valeur initiale justifiée.
- Confondre précision de l'algorithme et précision des mesures.
- Accepter une concentration négative ou supérieure à la quantité initiale.
- Construire A et B dans des ordres différents.

Validation rapide

Je sais : définir l'inconnue ; écrire $f(x) = 0$; repérer un intervalle ; expliquer une étape de dichotomie ; utiliser **bisect** et **newton** ; écrire $AX = B$; utiliser **solve** ; remplacer la solution dans le modèle ; contrôler son sens physique.

À retenir

Une solution numérique n'est exploitable que si elle répond à l'équation, appartient au domaine physique et reste cohérente avec la précision des données.